Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5

Obtained Marks: _____

Artificial Neural Networks

(Lab)

Task # 02

Submitted To: Mr. Hassaan Mujtaba

Student Name: Aleeya Fatima

Registration Number: 24108108

Code no.1:

```python
#  Task: Flood Prediction using Logistic Regression
#  Dataset: flood_dataset_classification.csv
#  Goal: Predict if a flood will occur (1 = Yes, 0 = No)
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Step 1: Read the dataset
data = pd.read_csv("flood_dataset_classification.csv")
print("Shape of data:", data.shape)
print(data.head())
# Step 2: Select features and target
features = ["Rainfall", "Elevation", "Slope", "distance"]
target = "occured"
X = data[features]
y = data[target]
# Step 3: Divide data into train (75%) and test (25%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=7)
# Step 4: Scale the values (logistic regression works better on scaled data)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Step 5: Build and train the model
clf = LogisticRegression()
clf.fit(X_train, y_train)
# Step 6: Evaluate the model
predictions = clf.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, predictions) * 100, 2), "%")
print("Confusion Matrix:\n", confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))
# Step 7: Function that gives the prediction for new values
def flood_check(rainfall, elevation, slope, distance):
    new_data = pd.DataFrame([[rainfall, elevation, slope, distance]], columns=features)
    new_data = scaler.transform(new_data)
    answer = clf.predict(new_data)[0]
    chance = clf.predict_proba(new_data)[0][1] * 100
    if answer == 1:
        return "Flood Likely (" + str(round(chance, 1)) + "% chance)"
    else:
        return "No Flood Expected (" + str(round(chance, 1)) + "% chance)"
#  Gradio App
import gradio as gr
with gr.Blocks() as flood_app:
    gr.Markdown("# Flood Prediction System")
```

```
    gr.Markdown("Enter the values below and click **Check Flood**")

    rainfall = gr.Number(label="Rainfall (mm)")
    elevation = gr.Number(label="Elevation (m)")
    slope = gr.Number(label="Slope")
    distance = gr.Number(label="Distance from River")

    btn = gr.Button("Check Flood")
    output = gr.Textbox(label="Result")

    btn.click(flood_check, inputs=[rainfall, elevation, slope, distance], outputs=output)
flood_app.launch()
#  Flask App  (running this part in a NEW cell)
from flask import Flask, request, render_template_string
web = Flask(__name__)
page = """
<html>
<body style="font-family: Arial; background-color: #e8f4fc; padding: 30px;">
    <h1>Flood Prediction System</h1>
    <form action="/predict" method="post">
        <p>Rainfall (mm): <input type="text" name="rainfall" required></p>
        <p>Elevation (m): <input type="text" name="elevation" required></p>
        <p>Slope: <input type="text" name="slope" required></p>
        <p>Distance from River: <input type="text" name="distance" required></p>
        <input type="submit" value="Check Flood">
    </form>
    <h2 style="color: darkblue;">{{ result }}</h2>
</body>
</html>
"""
@web.route("/")
def index():
    return render_template_string(page, result="")
@web.route("/predict", methods=["POST"])
def predict():
    rainfall = float(request.form["rainfall"])
    elevation = float(request.form["elevation"])
    slope = float(request.form["slope"])
    distance = float(request.form["distance"])
    result = flood_check(rainfall, elevation, slope, distance)
    return render_template_string(page, result=result)
web.run(port=5004)
```

Output:

```
Shape of data: (6237, 12)
   Disaster Type  Latitude  Longitude  Total Deaths  Total Affected  duration  \
0              0   52.6717    -0.2995         300.0          3000.0       0.0
1              0   35.6897   139.6920        1379.0         13790.0       0.0
2              0   39.9050   116.3910      100000.0       1000000.0       0.0
3              0   23.1288   113.2590           0.0       3000000.0       0.0
4              0   39.1467   117.2060           0.0        635000.0       0.0

   time     Rainfall  Elevation      Slope  distance  occured
0  1900  1383.125626       11.0   1.788207       0.0        1
1  1909  1383.125626       49.0  24.356508       0.0        1
2  1909   580.345856       55.0   8.374380       0.0        1
3  1912  2993.401777        7.0  12.917221       0.0        1
4  1913  1383.125626        3.0   1.513093       0.0        1
Accuracy: 70.71 %
Confusion Matrix:
 [[114 412]
 [ 45 989]]
              precision    recall  f1-score   support

           0       0.72      0.22      0.33       526
           1       0.71      0.96      0.81      1034

    accuracy                           0.71      1560
   macro avg       0.71      0.59      0.57      1560
weighted avg       0.71      0.71      0.65      1560

* Running on local URL:  http://127.0.0.1:7861
* To create a public link, set `share=True` in `launch()`.
```

App:



Explanation:

- In this task we predict whether a flood will occur at a location (1 = Yes, 0 = No). Logistic regression is used because the answer is a category (yes or no) rather than a number. It works out the probability of a flood and labels the case "flood" when that probability is above 50%.
- First, the flood dataset is loaded with pandas. Four columns are chosen as inputs: Rainfall, Elevation, Slope and Distance from the river. The column "occured" is the target we want to predict. The data is split so that 75% is used for training and 25% is kept for testing. The values are then scaled with StandardScaler, because columns like distance are in the thousands while slope is small, and scaling puts them all on the same level so the model learns fairly from each one.
- The model is trained on the training data and checked on the test data. It reaches about 71% accuracy. The confusion matrix shows how many floods and non-floods it predicted correctly and wrongly, and the classification report gives precision and recall for each class.
- Finally, a function called flood_check takes new values from the user and returns the prediction along with the flood chance in percent. This function is connected to two interfaces. The Gradio app (built with gr.Blocks) shows input boxes and a button inside the notebook. The Flask app is a web page with a form that sends the values to the /predict page and shows the result.

Code no.2:

```
#  Task: Flood Prediction using Logistic Regression
#  Dataset: flood_dataset_classification.csv
#  Goal: Predict if a flood will occur (1 = Yes, 0 = No)
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Step 1: Read the dataset
data = pd.read_csv("flood_dataset_classification.csv")
print("Shape of data:", data.shape)
print(data.head())
# Step 2: Select features and target
features = ["Rainfall", "Elevation", "Slope", "distance"]
target = "occured"
X = data[features]
y = data[target]
# Step 3: Divide data into train (75%) and test (25%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=7)
# Step 4: Scale the values (logistic regression works better on scaled data)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Step 5: Build and train the model
clf = LogisticRegression()
clf.fit(X_train, y_train)
# Step 6: Evaluate the model
predictions = clf.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, predictions) * 100, 2), "%")
print("Confusion Matrix:\n", confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))
# Step 7: Function that gives the prediction for new values
def flood_check(rainfall, elevation, slope, distance):
    new_data = pd.DataFrame([[rainfall, elevation, slope, distance]], columns=features)
    new_data = scaler.transform(new_data)
    answer = clf.predict(new_data)[0]
    chance = clf.predict_proba(new_data)[0][1] * 100
    if answer == 1:
        return "Flood Likely (" + str(round(chance, 1)) + "% chance)"
    else:
        return "No Flood Expected (" + str(round(chance, 1)) + "% chance)"
#  Gradio App
import gradio as gr
with gr.Blocks() as flood_app:
    gr.Markdown("# Flood Prediction System")
```

```
#  Gradio App
import gradio as gr
with gr.Blocks() as flood_app:
    gr.Markdown("# Flood Prediction System")
    gr.Markdown("Enter the values below and click **Check Flood**")

    rainfall = gr.Number(label="Rainfall (mm)")
    elevation = gr.Number(label="Elevation (m)")
    slope = gr.Number(label="Slope")
    distance = gr.Number(label="Distance from River")
    btn = gr.Button("Check Flood")
    output = gr.Textbox(label="Result")
    btn.click(flood_check, inputs=[rainfall, elevation, slope, distance], outputs=output)
flood_app.launch()
#  Flask App  (running this part in a NEW cell)
from flask import Flask, request, render_template_string
web = Flask(__name__)
page = """
<html>
<body style="font-family: Arial; background-color: #e8f4fc; padding: 30px;">
    <h1>Flood Prediction System</h1>
    <form action="/predict" method="post">
        <p>Rainfall (mm): <input type="text" name="rainfall" required></p>
        <p>Elevation (m): <input type="text" name="elevation" required></p>
        <p>Slope: <input type="text" name="slope" required></p>
        <p>Distance from River: <input type="text" name="distance" required></p>
        <input type="submit" value="Check Flood">
    </form>
    <h2 style="color: darkblue;">{{ result }}</h2>
</body>
</html>
"""
@web.route("/")
def index():
    return render_template_string(page, result="")
@web.route("/predict", methods=["POST"])
def predict():
    rainfall = float(request.form["rainfall"])
    elevation = float(request.form["elevation"])
    slope = float(request.form["slope"])
    distance = float(request.form["distance"])
    result = flood_check(rainfall, elevation, slope, distance)
    return render_template_string(page, result=result)
web.run(port=5004)
```

Output:

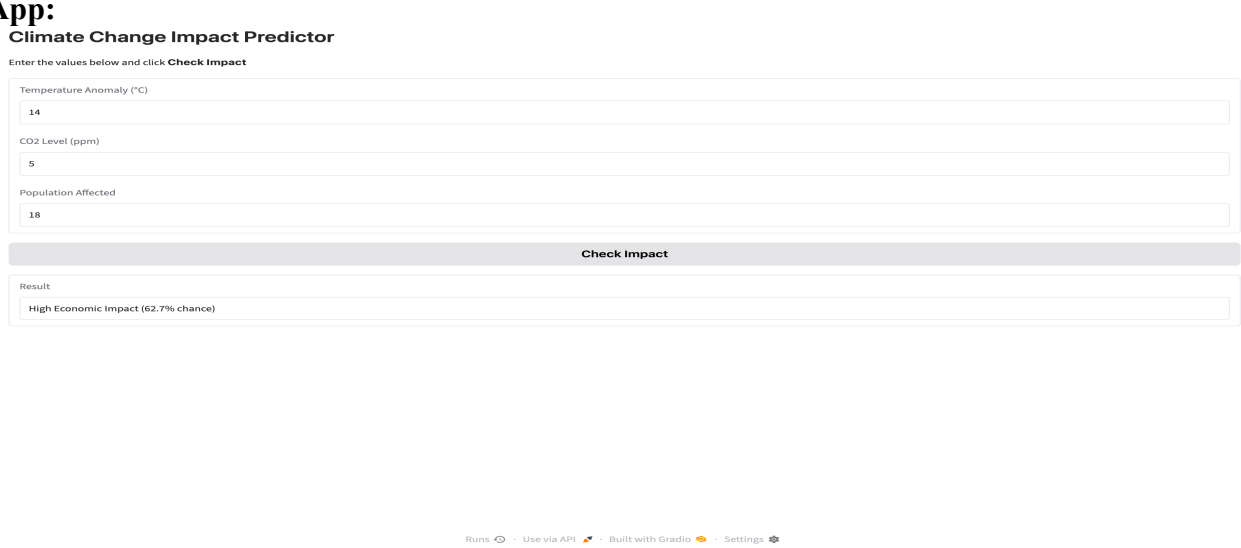
```
Shape of data: (6237, 12)
   Disaster Type  Latitude  Longitude  Total Deaths  Total Affected  duration  \
0              0   52.6717    -0.2995         300.0          3000.0       0.0
1              0   35.6897   139.6920        1379.0         13790.0       0.0
2              0   39.9050   116.3910      100000.0       1000000.0       0.0
3              0   23.1288   113.2590           0.0       3000000.0       0.0
4              0   39.1467   117.2060           0.0        635000.0       0.0

   time     Rainfall  Elevation      Slope  distance  occured
0  1900  1383.125626       11.0   1.788207       0.0        1
1  1909  1383.125626       49.0  24.356508       0.0        1
2  1909   580.345856       55.0   8.374380       0.0        1
3  1912  2993.401777        7.0  12.917221       0.0        1
4  1913  1383.125626        3.0   1.513093       0.0        1
Accuracy: 70.71 %
Confusion Matrix:
 [[114 412]
 [ 45 989]]
              precision    recall  f1-score   support

           0       0.72      0.22      0.33       526
           1       0.71      0.96      0.81      1034

    accuracy                           0.71      1560
   macro avg       0.71      0.59      0.57      1560
weighted avg       0.71      0.71      0.65      1560
```

App:



Explanation:

- In this task we predict whether the economic impact of a climate event will be High (1) or Low (0). The dataset has no ready-made yes/no column, so we create one. We find the median (middle value) of EconomicImpact_USD and label every row above it as High and the rest as Low. This turns the problem into a classification task.
- The inputs are Temperature Anomaly, CO2 Level and Population Affected. As in Task 1, the data is split 75/25, the values are scaled, and a logistic regression model is trained and tested with accuracy, a confusion matrix and a classification report.
- The accuracy is about 50%. This is because the columns in this dataset have no real relationship with each other, so there is no pattern for the model to learn. That is a limitation of the data, not a mistake in the code, and it is worth mentioning in your report.
- The impact_check function returns High or Low impact along with the probability. It is shown to the user through a Gradio app and a Flask web page, the same way as in Code no# 1.